Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

Факультет «Информатика и системы управления» (ИУ)

Кафедра «Информационная безопасность» (ИУ8)

Отчет

по научно-исследовательской работе студента

на тему Интеграция верификации соответствия TLA+ спецификации и исходного кода в систему распределенных вычислений

ФИО студента: Железцов Никита Владимирович

Группа: ИУ8-124-2026 Л.д.: 20У474

Специальность: 10.05.01 Компьютерная безопасность

Специализация: 10.05.01_01 Математические методы защиты информации

Научный руководитель НИРС доцент кафедры ИУ8 Медведев Николай Викторович

Работа выполнена _____ Н.В.Железцов
дата подпись студента И.О.Фамилия студента

Допуск к защите _____ Н.В.Медведев
дата подпись научного руководителя И.О.Фамилия научного руководителя

Отчет принят _____ Д.О.Левиев
дата подпись ответственного за НИРС И.О.Фамилия Ответственного за НИРС

Результаты защиты НИРС			
Дата	Балл	Подпись	ФИО

Москва
2025

РЕФЕРАТ

Отчёт содержит 35 стр., 3 рис., 6 источн., 2 прил.

Ключевые слова: TLA+, Raft, трассировка, валидация трасс, TLC, распределённые системы, интеграция.

Основная цель работы — интеграция формальной TLA+-спецификации и исполняемой распределённой системы за счёт встроенной трассировки и автоматической проверки трасс исполнения.

В работе реализован механизм инструментирования программы, формирующий трассы изменений состояния в формате NDJSON, а также программный конвейер их объединения и проверки средством TLC на соответствие TLA+-спецификации алгоритма Raft. Подход апробирован на собственной реализации распределённой системы вычислений с кластером консенсуса.

В ходе проверки показано, что предложенная интеграция позволяет выявлять расхождения между формальной моделью и реальным кодом, включая ошибки в логике выборов и обработки сообщений.

СОДЕРЖАНИЕ

РЕФЕРАТ 4
ВВЕДЕНИЕ 6
ОСНОВНАЯ ЧАСТЬ 9
1 Методика верификации 10
2 Архитектура распределённой системы 11
3 TLA+ спецификация алгоритма консенсуса Raft 14
3.1 Структура спецификации 14
3.2 Модель переходов 15
3.3 Проверяемые свойства 16
3.4 Проверка модели 17
4 Проверка соответствия реализации и спецификации 18
4.1 Инструментирование реализации 18
4.2 Инфраструктура запуска и проверки 19
4.3 Стратегия тестирования 21
4.4 Обнаруженные несоответствия 22
ЗАКЛЮЧЕНИЕ 25
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 27
ПРИЛОЖЕНИЕ А Методика верификации 28
ПРИЛОЖЕНИЕ Б Спецификация Raft на TLA+ 29

ВВЕДЕНИЕ

Распределённые системы являются основой современных вычислительных платформ, облачных сервисов, систем хранения данных и инфраструктуры обработки задач. Именно в таких системах особенно остро проявляются проблемы согласованности, отказоустойчивости и корректного взаимодействия независимых узлов. Даже небольшая ошибка в алгоритме координации может приводить к расхождению состояний реплик, потере результатов вычислений, некорректному выбору лидера или нарушению гарантий безопасности. Сложность анализа подобных ошибок обусловлена тем, что поведение распределённой системы формируется в условиях асинхронности, недетерминированной доставки сообщений, конкурентного выполнения и возможных сбоев отдельных процессов и сетевых соединений.

Для повышения надёжности распределённых алгоритмов всё шире применяются формальные методы. Одним из наиболее известных и практически значимых средств формального описания является язык спецификаций TLA+, предложенный Л. Лэмпортом [1]. Он позволяет задавать систему как множество состояний и допустимых переходов между ними, а также формулировать инварианты и свойства корректности, которые должны сохраняться при любом выполнении. Средство проверки моделей TLC, входящее в экосистему TLA+, делает возможным автоматическое исследование пространства состояний спецификации и выявление нарушений сформулированных свойств.

Однако наличие корректной спецификации ещё не означает корректность реальной программной реализации. Между формальной моделью и исходным кодом существует естественный разрыв: спецификация описывает поведение на абстрактном уровне, тогда как реализация включает сетевое взаимодействие, многопоточность, внутренние буферы, особенности библиотек и инфраструктурные детали. Именно на этом этапе могут появляться расхождения, которые не видны при проверке одной лишь модели. Поэтому важной практической задачей становится не только верификация спецификации, но и проверка того, что исполняемая программа действительно следует этой спецификации.

В данной работе рассматривается подход к валидации реализации распределённого алгоритма консенсуса Raft по формальной спецификации TLA+ на основе анализа трасс исполнения. Для этого реализация инструментируется таким образом, чтобы во время работы фиксировать события, соответствующие изменениям переменных спецификации. Полученные трассы затем сопоставляются со спецификацией при помощи TLC через специальную trace-обёртку. Такой подход позволяет проверять не абстрактную модель отдельно от программы, а реальное поведение запущенной системы, воспроизведённое в виде последовательности наблюдаемых переходов.

Актуальность темы исследования

Актуальность темы определяется тем, что традиционные методы тестирования распределённых систем ограниченно подходят для обнаружения ошибок, связанных с редкими межпоточными сценариями, особенностями сетевого взаимодействия и переходами между состояниями протокола. Модульные и интеграционные тесты обычно проверяют отдельные заранее предусмотренные случаи, тогда как множество фактически возможных сценариев в распределённой системе значительно шире. Особенно трудно воспроизводимыми являются ошибки, возникающие при специфическом порядке прихода сообщений, повторной доставке, смене лидера или конкуренции между параллельными действиями узлов.

Формальная спецификация в TLA+ позволяет строго описать корректное поведение алгоритма, однако без сопоставления со следами реального исполнения остаётся открытым вопрос, действительно ли программа реализует именно эту модель. Следовательно, необходим метод, который связывает спецификацию и код на уровне наблюдаемого поведения системы. Перспективным решением является модельная валидация трасс исполнения, при которой последовательность событий, зафиксированных в программе, проверяется на соответствие множеству переходов, разрешённых формальной спецификацией [2]. Такой подход позволяет обнаруживать не только дефекты реализации, но и ошибки инструментирования, расхождения в уровне атомарности действий и слишком сильные или, наоборот, неполные предположения в самой спецификации.

Практическая значимость такого подхода особенно высока для алгоритмов консенсуса, поскольку именно они лежат в основе отказоустойчивых сервисов, реплицированных хранилищ и систем координации. Алгоритм Raft широко применяется как более понятная альтернатива Paxos и используется в ряде промышленных систем. Поэтому разработка и апробация метода проверки соответствия реализации Raft его формальной спецификации представляет как теоретический, так и прикладной интерес. Подобные подходы к использованию формальных методов для анализа реальных распределённых систем уже доказали свою эффективность в индустрии [3].

Цель и задачи работы

Целью данной работы является разработка и экспериментальная апробация метода проверки соответствия реализации распределённого алгоритма консенсуса Raft его формальной спецификации на языке TLA+ на основе трасс исполнения программы.

Для достижения поставленной цели были сформулированы следующие задачи:

а) Разработать инструменты подготовки, объединения и проверки трасс исполнения относительно спецификации с использованием TLC.

б) Провести экспериментальную проверку реализации системы распределенных вычислений, сопоставить реальные трассы со спецификацией и проанализировать обнаруженные расхождения.

ОСНОВНАЯ ЧАСТЬ

В основной части работы последовательно рассматриваются как теоретические, так и практические аспекты предложенного подхода. Сначала описывается методика верификации, основанная на сопоставлении трасс исполнения реализации с формальной спецификацией на языке TLA+. Затем приводится архитектура разработанной распределённой системы и формальная спецификация алгоритма консенсуса Raft. Завершает основную часть описание процедуры проверки соответствия между реализацией и спецификацией, включая используемые инструменты, ход экспериментов и выявленные расхождения.

1 Методика верификации

Предлагаемая методика верификации основана на сопоставлении трасс исполнения программы с формальной спецификацией, написанной на языке TLA+. Схема подхода представлена в Приложении А.

Исходными артефактами являются программная реализация алгоритма и формальная спецификация этого алгоритма на языке TLA+. Спецификация задаёт множество допустимых состояний системы и переходов между ними, однако не содержит сведений о конкретной реализации. Реализация, в свою очередь, выполняется в распределённой среде, где возможны различные порядки доставки сообщений и недетерминированные сценарии выполнения.

Для сопоставления поведения реализации со спецификацией используется инструментирование кода. В реализацию внедряется библиотека трассировки, которая фиксирует события, соответствующие переходам спецификации, а также изменения переменных спецификации. При выполнении системы каждый процесс (координатор и участники) формирует собственную трассу в формате JSON, содержащую последовательность операций обновления логических переменных модели. Полученные локальные трассы агрегируются в единый глобальный файл, отражающий наблюдаемую последовательность изменений состояния всей распределённой системы. Таким образом, формируется абстрактное представление исполнения, выраженное в терминах переменных TLA+, а не в терминах низкоуровневых операций кода.

Для проверки корректности трассы используется дополнительная спецификация, построенная на основе исходной TLA+ спецификации. Она расширяет её таким образом, чтобы поведение модели было ограничено наблюдаемой последовательностью изменений. Иными словами, трасса интерпретируется как частичное ограничение на допустимые переходы модели.

Далее средство модельной проверки TLC используется для проверки существования поведения спецификации, согласованного с данной трассой. Если TLC подтверждает корректность, это означает, что наблюдаемое исполнение допустимо спецификацией. В противном случае обнаруживается расхождение между реализацией и моделью, и TLC предоставляет информацию, позволяющую локализовать проблему.

2 Архитектура распределённой системы

Высокоуровневая архитектура системы основана на разделении функций координации и непосредственного выполнения вычислений. В системе выделяются три основных компонента: клиент, кластер узлов консенсуса Raft и набор вычислительных узлов (рабочий). Такое разделение соответствует классической модели разделения *control plane* и *data plane*: кластер Raft отвечает за согласование, хранение и изменение глобального состояния, а рабочие узлы исполняют пользовательские задачи, не принимая решений, влияющих на консистентность всей системы [4]. Общая схема взаимодействия показана на рис. 1.

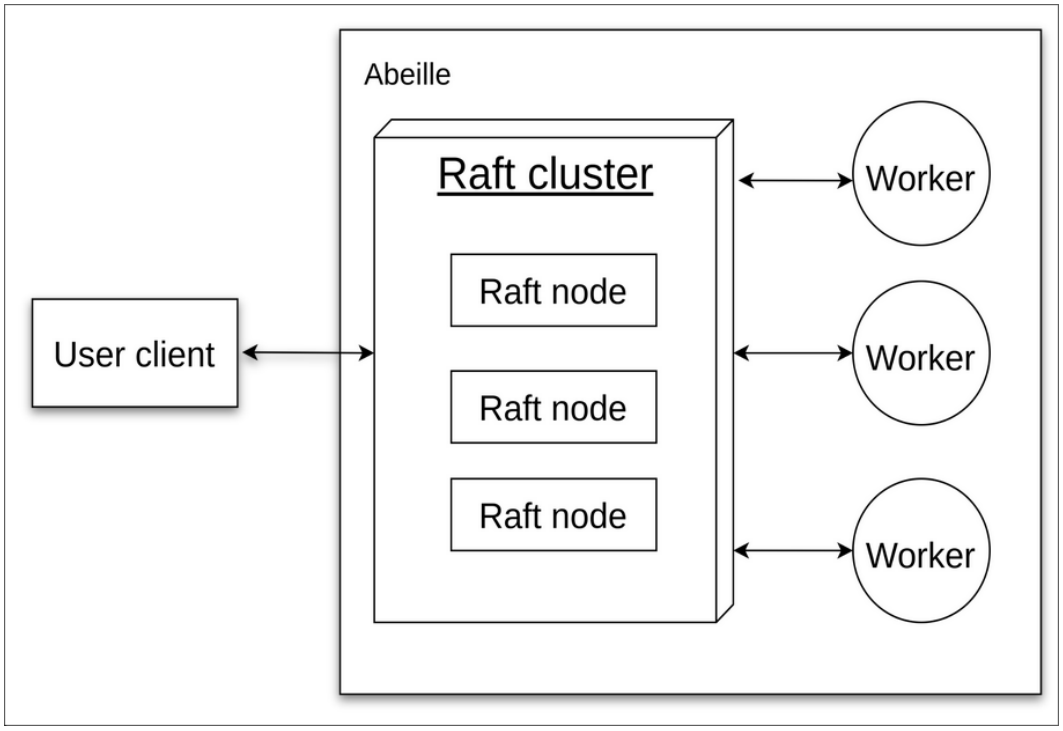

Рисунок 1 – Высокоуровневая архитектура распределённой системы

Клиент взаимодействует только с кластером Raft. Это означает, что любая операция, изменяющая состояние системы, сначала попадает на один из узлов кластера, при необходимости перенаправляется лидеру, затем записывается в лог Raft, реплицируется на большинство узлов и только после этого считается подтверждённой. Такой подход позволяет использовать кластер как единственный источник истины о состоянии распределённой системы. В частности, через него фиксируются постановка задач, назначение задачи

конкретному рабочему узлу, изменение статуса выполнения и сохранение итогового результата. Благодаря этому система получает детерминированный порядок событий и устойчивость к повторам запросов, сетевым задержкам и частичным отказам.

Кластер Raft играет роль координационного центра. Его узлы поддерживают реплицированную машину состояний, принимают клиентские запросы, выполняют их валидацию и принимают решения о диспетчеризации задач. После коммита новой операции в машине состояний лидер может безопасно передать задание вычислительному узлу. Аналогично и обратный поток данных организован через кластер: рабочий не отправляет результат напрямую клиенту, а возвращает его в кластер, где он также проходит через запись в лог и коммит. Лишь после этого результат становится наблюдаемым для клиента. Таким образом, даже при отказе отдельного узла или смене лидера система сохраняет согласованность состояния и не допускает появления «локально известных», но глобально не подтверждённых результатов.

Вычислительные узлы образуют исполнительный контур системы. Их задача состоит в получении заданий от кластера, выполнении пользовательского кода и передаче результатов обратно. В предлагаемой архитектуре рабочие узлы остаются максимально простыми: они не участвуют в алгоритме консенсуса, не координируются между собой и не принимают решений о глобальном состоянии системы. Это делает их удобными для масштабирования: увеличение числа рабочих повышает вычислительную пропускную способность системы без усложнения протокола согласования. Иными словами, консенсусный контур остаётся компактным и управляемым, а исполнительный контур — эластичным.

Принципиальным архитектурным решением является отказ от прямого взаимодействия между клиентом и рабочими узлами, а также отказ от горизонтального взаимодействия между самими рабочими узлами. Такое ограничение уменьшает связанность системы и упрощает обеспечение корректности. Во-первых, все проверки аутентификации, авторизации, квот и политик исполнения могут быть сосредоточены в одном месте — в кластере Raft. Во-вторых, единая точка сериализации операций устраняет риск

расхождения состояния при сетевых разделениях и гонках между несколькими исполнителями. В-третьих, вычислительные узлы можно проектировать как практически без состояния или со слабо связанным локальным состоянием, что существенно упрощает их замену, перезапуск и масштабирование.

С точки зрения отказоустойчивости система ориентирована на модель crash-stop/crash-recovery и частично синхронную сеть. Безопасность кластера обеспечивается алгоритмом Raft: два различных значения не могут быть закоммичены в одной и той же позиции лога, а после выбора нового лидера система продолжает работу с согласованного состояния [5]. При отказе отдельного рабочего узла целостность системы не нарушается, поскольку состояние задачи уже зафиксировано в машине состояний и может быть использовано для повторного назначения. При отказе лидера выполняется переизбрание, и после достижения кворума обслуживание запросов продолжается. Для кластера из N узлов консенсуса система сохраняет работоспособность при отказе до $\lfloor (N-1)/2 \rfloor$ узлов, что соответствует стандартной кворумной гарантии [6].

Рассматривались и альтернативные варианты построения системы: прямой доступ клиента к рабочим узлам, объединение ролей вычислительных и консенсусных узлов, а также более децентрализованные схемы взаимодействия исполнителей. Однако такие решения либо увеличивают поверхность ошибок и сложность сопровождения, либо ухудшают масштабируемость за счёт роста числа участников консенсуса. Выбранная архитектура, напротив, позволяет чётко локализовать инварианты безопасности внутри кластера Raft и одновременно сохранить простоту вычислительного контура. Поэтому она является наиболее подходящей для системы распределённых вычислений, в которой критически важны согласованность, отказоустойчивость и предсказуемое масштабирование.

3 TLA+ спецификация алгоритма консенсуса Raft

Для формального описания и последующей проверки корректности алгоритма консенсуса в работе использована спецификация на языке TLA+. Разработанная спецификация моделирует основные элементы поведения кластера: локальные состояния узлов, обмен сообщениями между ними, накопление и согласование записей журнала, а также продвижение подтверждённых записей в машину состояний. При этом в спецификации сознательно опущены низкоуровневые детали программной реализации, такие как потоки исполнения, особенности gRPC, структура классов или формат хранения данных в памяти. Вместо этого фиксируется именно та семантика, которая существенна для проверки корректности алгоритма на уровне распределённого взаимодействия.

Следует подчеркнуть, что спецификация используется в работе в двух ролях. С одной стороны, она выступает как самостоятельная формальная модель Raft, позволяющая проверить базовые свойства безопасности с помощью TLC. С другой стороны, она служит эталоном для сопоставления с реальной реализацией: именно в терминах этой модели впоследствии интерпретируются трассы исполнения инструментированной программы. Поэтому важно, чтобы спецификация была достаточно абстрактной для удобной проверки, но при этом достаточно точной, чтобы по ней можно было судить о корректности поведения реализованного кода.

Полный текст спецификации приведён в приложении Б. В данном разделе рассматриваются её структура, основные переходы системы, проверяемые свойства и порядок модельной проверки.

3.1 Структура спецификации

Модуль `Raft` расширяет стандартные модули `Naturals`, `FiniteSets`, `Sequences` и `TLC`. В спецификации задаются константы, определяющие множество серверов `Server`, множество возможных значений `Value`, роли узлов (`Follower`, `Candidate`, `Leader`), типы сообщений протокола (`RequestVoteRequest`, `RequestVoteResponse`, `AppendEntriesRequest`, `AppendEntriesResponse`), а также ограничения на число выборов и рестартов (`MaxElections`, `MaxRestarts`). Эти параметры фиксируют модель кластера и позволяют ограничить пространство состояний при проверке.

Состояние системы описывается набором глобальных и локальных переменных. Глобальная переменная `messages` моделирует сеть как мультимножество сообщений, находящихся “в полёте”. Дополнительно введены вспомогательные переменные `acked`, `electionCtr` и `restartCtr`, которые используются для проверки свойств модели и ограничения числа переходов. Для каждого сервера задаются стандартные переменные Raft: `currentTerm`, `state`, `votedFor`, журнал `log`, индекс коммита `commitIndex`, множество уже полученных голосов `votesGranted`, а также лидерские структуры `nextIndex`, `matchIndex` и `pendingResponse`. Последняя переменная позволяет запретить одновременную отправку нескольких `AppendEntries` одному и тому же узлу и тем самым уменьшить число межсостояний в модели.

Начальное состояние задаётся оператором `Init`. В нём все узлы находятся в состоянии `Follower`, имеют начальный терм, пустой журнал, нулевой `commitIndex`, не голосовали ни за одного кандидата и не имеют недоставленных сообщений. Таким образом, `Init` описывает корректную исходную конфигурацию кластера до начала выборов и репликации.

3.2 Модель переходов

Динамика протокола задаётся оператором `Next`, представляющим собой дизъюнкцию всех допустимых действий. В модели отражены основные переходы Raft.

Действие `RequestVote(i)` описывает начало выборов: сервер i переходит в состояние кандидата, увеличивает терм, голосует за себя и рассылает запросы `RequestVoteRequest` другим узлам. Если кандидат получает большинство голосов, срабатывает `BecomeLeader(i)`: узел становится лидером и инициализирует структуры `nextIndex` и `matchIndex` для репликации журнала.

Репликация журнала задаётся действием `AppendEntries(i, j)`, в котором лидер i отправляет последователю j сообщение `AppendEntriesRequest`. В модели за один шаг передаётся не более одной записи журнала, что упрощает проверку, но сохраняет основную семантику алгоритма. Ответы на такие запросы обрабатываются переходом `HandleAppendEntriesResponse`, который либо обновляет `matchIndex` и `nextIndex` при успехе, либо уменьшает `nextIndex` при обнаружении расхождения журналов.

Получение и обработка голосов задаются действиями `HandleRequestVoteRequest` и `HandleRequestVoteResponse`. В них формализуются стандартные правила Raft: голос выдаётся только если кандидат не уступает получателю по “свежести” журнала и если узел ещё не голосовал в данном терме.

Отдельное действие `UpdateTerm` отвечает за переход узла в новый терм при получении сообщения с большим значением `term`. В этом случае узел становится `Follower` и сбрасывает локальное решение о голосовании. Таким образом, в спецификации явно выделен механизм, обеспечивающий монотонный рост термов и корректную реакцию на появление более “свежего” состояния кластера.

Продвижение подтверждённого индекса описывает действие `AdvanceCommitIndex`. Лидер вычисляет максимальный индекс, который реплицирован на кворум узлов и относится к его текущему терму, после чего перемещает `commitIndex` вперёд. Это соответствует ключевому правилу Raft, запрещающему коммит записей старых термов только на основании большинства подтверждений.

Наконец, в модели присутствует переход `Restart(i)`, который имитирует перезапуск узла. При рестарте сбрасываются временные структуры состояния (например, лидерские и кандидатские переменные), однако сохраняются персистентные данные, прежде всего `currentTerm`, `votedFor` и `log`. Такое поведение соответствует модели аварийного завершения и восстановления, заложенной в алгоритме Raft.

3.3 Проверяемые свойства

На уровне модели проверяются прежде всего свойства безопасности. В спецификации заданы два основных инварианта.

Инвариант `NoLogDivergence` утверждает, что если две реплики считают некоторую префиксную часть журнала подтверждённой, то эти префиксы совпадают. Иначе говоря, у двух узлов не может быть разных значений в одной и той же уже зафиксированной позиции журнала. Это выражает фундаментальное свойство согласованности реплицированного лога.

Инвариант `LeaderHasAllAckedValues` формализует требование, согласно которому любое значение, уже подтверждённое клиенту, должно присутствовать в журнале любого актуального лидера. Тем самым исключается потеря данных, которые были признаны системой завершёнными.

Помимо инвариантов, спецификация определяет полную систему переходов `Spec == Init / [][Next]_vars`. Для исключения пустых шагов также используется условие слабой справедливости `WF_vars(Next)`, что позволяет анализировать поведение системы без искусственного застревания в неизменных состояниях.

3.4 Проверка модели

Проверка спецификации выполнялась с помощью средства `TLC`. В конфигурации модели задавались конкретные множества серверов и значений, а также ограничения `MaxElections` и `MaxRestarts`, позволяющие сделать перебор конечным. В ходе проверки анализировались все достижимые состояния в рамках выбранных параметров, после чего автоматически проверялись инварианты безопасности.

Таким образом, TLA+-спецификация позволила формально зафиксировать поведение алгоритма Raft на уровне переходной системы и проверить, что в рассматриваемой модели не нарушаются ключевые свойства согласованности журнала и сохранности подтверждённых данных.

4 Проверка соответствия реализации и спецификации

После построения TLA+-спецификации алгоритма Raft следующим шагом стала проверка того, насколько фактическое поведение реализации совпадает с поведением, допускаемым формальной моделью. В данной работе использовался не подход “доказать корректность исходного кода напрямую”, а более практичная и воспроизводимая схема валидации трасс исполнения: программа инструментировалась таким образом, чтобы во время работы порождать машиночитаемую трассу абстрактных событий, после чего эта трасса проверялась модельным проверяющим TLC на соответствие спецификации `Raft.tla` (см. Приложение Б).

Такой подход оказался особенно полезен по двум причинам. Во-первых, он позволяет проверять не отдельные локальные свойства кода, а целостные сценарии исполнения распределённой системы, включая выборы лидера, обмен RPC-сообщениями и переходы между ролями узлов. Во-вторых, он даёт строгий критерий несоответствия: если TLC не может сопоставить очередной шаг трассы с допустимым действием спецификации, значит либо реализация действительно ведёт себя не так, как предписывает модель, либо ошибка содержится в инструментировании или вспомогательной обвязке проверки. Поэтому существенная часть работы заключалась не только в запуске TLC, но и в аккуратной организации всей цепочки “реализация → трасса → абстрактная модель”.

4.1 Инструментирование реализации

Для проверки соответствия в реализацию Raft был встроен отдельный механизм трассировки, задача которого состояла не в обычном логировании отладочных сообщений, а в фиксации именно тех изменений, которые имеют смысл с точки зрения спецификации. Иными словами, трассировщик должен был отражать не внутренние детали реализации, а переходы абстрактной модели: отправку и получение сообщений, изменение терма, переходы между ролями `Follower/Candidate/Leader`, учёт голосов, обновление индексов репликации и т.д.

Инструментирование было выполнено на уровне класса `RaftConsensus`. В ключевых точках алгоритма были добавлены вызовы специального объекта `RaftTracer`. Этот объект формировал записи трассы в формате NDJSON, по одной JSON-строке на событие. Каждая запись содержала:

- имя абстрактного события, например `RequestVote`, `BecomeLeader`, `AppendEntries`, `HandleRequestVoteRequest`, `HandleAppendEntriesResponse`, `AcceptAppendEntriesRequest`, `RejectAppendEntriesRequest`, `UpdateTerm`;
- набор элементарных операций над переменными модели: `Update`, `Add`, `Clear`, `AddElement`, `AddElementToBag`, `RemoveElementFromBag`;
- идентификатор процесса-источника, используемый только для отладки и корреляции событий из разных процессов;
- при необходимости — дополнительные аргументы события (`event_args`), например идентификатор сервера.

Особенно важным оказалось то, что одно действие реализации не всегда соответствует одному действию TLA+-модели. Например, при получении сообщения с большим термом реализация сначала должна логически выполнить действие `UpdateTerm`, а уже затем — обработать само сообщение (`HandleRequestVoteRequest` или `AcceptAppendEntriesRequest`). Именно поэтому трассировщик добавлялся не “на уровень RPC-обработчика целиком”, а в конкретные точки изменения абстрактного состояния. Без этого TLC не смог бы сопоставить трассу с последовательностью допустимых переходов модели.

Каждый процесс записывал трассу в отдельный файл. При запуске узла путь к файлу передавался через переменную окружения `TLAC_TRACE_FILE`. Дополнительно через `TLAC_CLOCK_FILE` передавался путь к общему файлу логических часов, который использовался для согласованного упорядочивания событий между несколькими процессами. Это было необходимо потому, что при параллельной работе узлов простого порядка строк в отдельных файлах недостаточно: для корректной проверки требуется единая линейная трасса.

4.2 Инфраструктура запуска и проверки

Для автоматизации экспериментов был собран отдельный набор инструментов, образующий полноценный конвейер проверки.

Скрипт `run_impl.py` отвечал за запуск кластера. Он читал конфигурацию запуска, поднимал нужное число узлов, выставлял переменные окружения трассировки, задавал рабочие каталоги и управлял временем жизни процессов.

Скрипт `pipeline.py` управлял всей процедурой целиком. Он выполнял следующие шаги:

а) загружал конфигурацию запуска;
б) выбирал один из кластерных сценариев;
в) формировал активные конфигурационные файлы для реализации и для TLC;
г) очищал старые трассы, снимки состояния и файл логических часов;
д) запускал реализацию через `run_impl.py`;
е) после завершения прогона собирал все порождённые трассы;
ж) объединял их в один итоговый NDJSON-файл;
з) запускал TLC на спецификации `RaftTrace.tla`.

Объединение трасс выполнял отдельный модуль `trace_merger.py`. Он собирал все фрагменты трасс, сортировал их по метаданным логических часов и удалял служебную информацию, не участвующую в семантике модели. На выходе получался единый файл `trace.ndjson`, который затем подавался на вход TLC.

Сама проверка строилась не на спецификации `Raft.tla` напрямую, а через обёртку `RaftTrace.tla`. Эта обёртка использовала шаблон `TraceSpec.tla` и интерпретировала записи NDJSON как последовательность наблюдаемых переходов системы. Идея состояла в следующем: TLC стартует из состояния `Init` и пытается шаг за шагом сопоставить очередную запись трассы с каким-либо допустимым действием спецификации. Если такое действие существует, проверка продолжается. Если нет, TLC останавливается и печатает первый шаг трассы, который не удаётся встроить в допустимое поведение модели.

Именно это свойство делало инструмент очень удобным для локализации ошибок. Вместо абстрактного сообщения “инвариант нарушен” получался вполне конкретный ответ: например, “не удалось сопоставить событие `RequestVote`” или “не удалось сопоставить событие `UpdateTerm`”. Далее можно было смотреть на содержимое соответствующей записи NDJSON и сравнивать её с тем, какие предусловия и эффекты имеет соответствующее действие в `Raft.tla`.

4.3 Стратегия тестирования

На практике выяснилось, что для надёжной локализации расхождений одного прогона полного трёхузлового кластера недостаточно. Если сразу запускать максимально сложный сценарий, трасса быстро разрастается, а точка первого несоответствия может оказаться следствием более ранней ошибки, замаскированной дальнейшими переходами. Поэтому проверка велась поэтапно.

Сначала использовались кластеры минимального размера: из одного, двух и трёх узлов. Для этого конфигурация запуска была переработана так, чтобы один и тот же набор инструментов мог поднимать разные подмножества узлов: `single-raft-0`, `single-raft-1`, `single-raft-2`, `pair-raft-0-1`, `pair-raft-0-2`, `pair-raft-1-2` и `triple-raft-0-1-2`. Такой подход позволял сначала проверить самые простые режимы, а затем постепенно увеличивать сложность сценария.

Кроме того, перед тем как считать очередное TLC-несоответствие реальным дефектом реализации, отдельно проверялась корректность самой инфраструктуры проверки. В ходе работы было обнаружено несколько проблем именно в обвязке: например, необходимость записывать “активные” конфигурационные файлы не в формате красиво отформатированного JSON, а строго в виде одной JSON-строки, иначе модуль `Json` внутри TLC не мог корректно десериализовать вход; или необходимость явно учитывать, что трассы одного узла могут записываться в файлы вида `raft-0.part000.ndjson`, `raft-0.part001.ndjson` и т.д. Лишь после устранения подобных проблем стало возможным уверенно интерпретировать ошибки TLC как ошибки самой реализации.

4.4 Обнаруженные несоответствия

В результате проверки были подтверждены два реальных расхождения реализации со спецификацией `Raft.tla`. Оба они воспроизводились на трассах, оба приводили к остановке TLC на конкретном шаге, и для обоих удалось локализовать причину непосредственно в исходном коде реализации.

Несовпадение начального значения `currentTerm`

Первое расхождение было обнаружено уже на самых ранних шагах исполнения, в момент первого запуска выборов. В спецификации начальные значения серверных переменных задаются следующим образом: каждый сервер стартует в состоянии `Follower`, с пустым журналом, с `votedFor = Nil` и с `currentTerm = 1`. Следовательно, первое действие `RequestVote(i)` в модели переводит узел из терма 1 в терм 2.

В реализации же поле `current_term_` было инициализировано значением `0`. Из-за этого первый запуск выборов переводил узел не в терм 2, а в терм 1. На уровне “обычной” отладки это могло показаться безобидной деталью: все узлы всё равно стартуют синхронно, а относительный порядок термов как будто сохраняется. Однако для формальной спецификации это существенное расхождение, потому что стартовое состояние модели фиксировано, и вся дальнейшая траектория строится относительно него.

Практически ошибка проявлялась так: TLC останавливался уже на первом событии `RequestVote`, так как в трассе кандидат переходил в `currentTerm = 1`, тогда как спецификация ожидала `currentTerm = 2`. Иными словами, первое же наблюдаемое действие реализации не могло быть встроено в допустимое поведение модели.

Это расхождение важно не только потому, что “не совпадает цифра”. Начальный терм — фундаментальная часть состояния Raft. Если реализация и спецификация различаются уже в стартовой точке, все последующие сравнения термов, условия `UpdateTerm`, выборы лидера и правила обработки RPC становятся смещёнными. Поэтому данная проблема была устранена в первую очередь: после приведения начального значения `current_term_` к тому, что ожидает спецификация, трасса стала проходить дальше, и TLC смог выявить уже следующее, более содержательное расхождение.

Отправка `RequestVote` самому себе

Второе подтверждённое несоответствие касалось состава множества peer-ов в реализации. Согласно спецификации, действие `RequestVote(i)` рассылает запросы `RequestVoteRequest` только серверам из множества `Server \{i}`. Это соответствует стандартной логике Raft: кандидат учитывает собственный голос локально, через `votedFor[i] = i` и `votesGranted[i] = {i}`, и не должен дополнительно отправлять RPC самому себе (см. рис. 2.

```
Starting... (2026-04-09 22:22:59)
Computing initial states...
Finished computing initial states: 1 distinct state generated at 2026-04-09 22:22:59.
<<"Failed matching the trace to (a prefix of) a behavior:", [messages ⟼ <<[op ⟼ "AddElementToBag", args ⟼ <<[mtype ⟼
 "RequestVoteRequest", mterm ⟼ 2, mdest ⟼ "127.0.0.1:50050", mlastLogTerm ⟼ 0, mlastLogIndex ⟼ 0, msource ⟼ "127.0
.0.1:50051"]>>, path ⟼ <<>>], [op ⟼ "AddElementToBag", args ⟼ <<[mtype ⟼ "RequestVoteRequest", mterm ⟼ 2, mdest ⟼
> "127.0.0.1:50051", mlastLogTerm ⟼ 0, mlastLogIndex ⟼ 0, msource ⟼ "127.0.0.1:50051"]>>, path ⟼ <<>>], [op ⟼ "Add
ElementToBag", args ⟼ <<[mtype ⟼ "RequestVoteRequest", mterm ⟼ 2, mdest ⟼ "127.0.0.1:50052", mlastLogTerm ⟼ 0, mla
stLogIndex ⟼ 0, msource ⟼ "127.0.0.1:50051"]>>, path ⟼ <<>>]>>, electionCtr ⟼ <<[op ⟼ "Add", args ⟼ <<1>>, path
⟼ <<>>]>>, currentTerm ⟼ <<[op ⟼ "Update", args ⟼ <<2>>, path ⟼ <<"127.0.0.1:50051">>]>>, state ⟼ <<[op ⟼ "Upd
ate", args ⟼ <<"Candidate">>, path ⟼ <<"127.0.0.1:50051">>]>>, votedFor ⟼ <<[op ⟼ "Update", args ⟼ <<"127.0.0.1:50
051">>, path ⟼ <<"127.0.0.1:50051">>]>>, votesGranted ⟼ <<[op ⟼ "Clear", args ⟼ <<>>, path ⟼ <<"127.0.0.1:50051">>
], [op ⟼ "AddElement", args ⟼ <<"127.0.0.1:50051">>, path ⟼ <<"127.0.0.1:50051">>]>>, event_args ⟼ <<"127.0.0.1:5005
1">>, event ⟼ "RequestVote", logger ⟼ "5f12fc84-23a5-69d7-0006-00006cf9c1d2"], "TLA+ debugger breakpoint hit count 2">>
 FALSE
Error: Assumption line 63, col 5 to line 66, col 84 of module TraceSpec is false.
Model checking completed. No error has been found.
  Estimates of the probability that TLC did not check all reachable states
  because two distinct states had the same fingerprint:
  calculated (optimistic):  val = 0.0
1 states generated, 1 distinct states found, 0 states left on queue.
```

Рисунок 2 – Нахождение расхождения при отправке запроса самому себе

Однако на трассах реализации было зафиксировано, что узел действительно порождает сообщение `RequestVoteRequest`, у которого адрес источника совпадает с адресом назначения. Это означало, что собственный адрес сервера ошибочно оставался в пуле peer-соединений, и кандидат начинал отправлять `RequestVote` самому себе как обычному соседу.

С точки зрения соответствия спецификации это уже само по себе является ошибкой: TLA+-модель такого шага не допускает. Но важнее то, что в реализации это могло искажать саму логику выборов. Собственный голос уже был учтён локально в момент перехода в состояние кандидата. Если после этого узел ещё и получал успешный `RequestVoteResponse` от самого себя, тот же голос мог быть учтён повторно — теперь уже как голос от peer-а. В предельном случае это позволяло кандидату достигнуть “кворума” раньше, чем будут получены реальные голоса от других узлов кластера.

Именно трассовая валидация позволила увидеть этот дефект в явном виде. В обычных журнальных логах можно было заметить только то, что “узел начал выборы” и “затем стал лидером”, но не было бы строгого критерия, по которому можно утверждать, что один из RPC-адресатов вообще не должен был существовать. Здесь же TLC останавливался на событии `RequestVote`, содержащем в списке исходящих сообщений self-RPC, и тем самым точно указывал на причину несовместимости.

Локализация причины показала, что проблема находится не в модели и не в трассировщике, а в логике формирования списка peer-ов: собственный адрес узла удалялся из этого списка некорректно, в результате чего self-peer сохранялся в пуле соединений. После исправления этой ошибки трассы, относящиеся к фазе выборов, стали существенно ближе к поведению, допускаемому спецификацией.

ЗАКЛЮЧЕНИЕ

В рамках данной работы была спроектирована и реализована распределённая система вычислений, в основе которой лежит кластер узлов, использующих алгоритм консенсуса Raft для поддержания согласованного состояния. Предложенная архитектура разделяет систему на контур координации и контур исполнения: кластер Raft отвечает за приём клиентских запросов, репликацию и фиксацию изменений состояния, а вычислительные узлы выполняют пользовательские задачи и возвращают результаты через согласованный серверный контур. Такое разделение позволило совместить требования к отказоустойчивости, детерминированности и масштабируемости.

В работе была детально описана архитектура основных компонентов системы: Raft-узлов, клиентской части и вычислительных узлов. Для реализации сетевого взаимодействия был использован стек `gRPC + Protocol Buffers`, обеспечивающий строгую типизацию интерфейсов и удобство сопровождения. Отдельное внимание было уделено реализации журнала Raft, машины состояний, механизма снимков данных, серверных сервисов и подсистемы диспетчеризации задач. В результате была получена рабочая программная система, поддерживающая распределённую постановку и выполнение вычислительных задач.

Существенной частью работы стало формальное описание алгоритма консенсуса на языке TLA+. Разработанная спецификация охватывает основные элементы Raft: состояния узлов, термы, голосование, репликацию журнала, продвижение `commitIndex`, обработку RPC-сообщений и ключевые инварианты безопасности. Формальная модель позволила зафиксировать не только ожидаемое поведение системы, но и те свойства, нарушение которых недопустимо в корректной реализации.

Наиболее важным результатом работы стала разработка и практическое применение подхода к проверке соответствия реализации и спецификации. Для этого реализация была инструментирована встроенным трассировщиком, фиксирующим абстрактные события исполнения в формате NDJSON. Далее был построен автоматизированный конвейер проверки, включающий запуск кластера, сбор и объединение трасс, а также их последующую

валидацию модельным проверяющим TLC через обёртку `RaftTrace.tla`. Тем самым формальная спецификация использовалась не только как средство описания алгоритма, но и как практический критерий корректности реальной реализации.

Проведённая проверка показала, что связка “инструментированная реализация — трасса исполнения — TLA+-проверка” является эффективным способом валидации распределённого кода. В отличие от обычных модульных и интеграционных тестов, такой подход сопоставляет наблюдаемое поведение программы с полным множеством переходов, разрешённых спецификацией, и позволяет выявлять ошибки, проявляющиеся только при определённых межпоточных и сетевых сценариях. С помощью разработанного инструментария были обнаружены и локализованы два подтверждённых расхождения реализации со спецификацией: неверное начальное значение терма и наличие self-peer в рассылке `RequestVote`. Оба дефекта затрагивали базовую логику алгоритма Raft, что подтверждает практическую полезность выбранного метода проверки.

Таким образом, поставленные в работе задачи были выполнены: разработана архитектура распределённой системы вычислений, реализован её программный прототип, построена формальная спецификация ключевого алгоритма консенсуса и создана инфраструктура для проверки соответствия реализации этой спецификации. Полученные результаты показывают, что сочетание инженерной реализации и формальных методов позволяет не только повысить доверие к корректности системы, но и использовать спецификацию как рабочий инструмент анализа и отладки.

Перспективы дальнейшего развития работы связаны прежде всего с расширением разработанной библиотеки трассировки и переносом предложенного подхода на более широкий круг систем. В ближайших планах находится поддержка новых языков программирования в библиотеке трассировки; минимальной целевой точкой является добавление языка Lua, что особенно важно для встраивания трассировки в СУБД Tarantool. Такая интеграция позволит применять подход “инструментированная реализация — трасса исполнения — формальная проверка” не только к учебным или исследовательским реализациям распределённых алгоритмов, но и к промышленным системам хранения и обработки данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Lamport L. Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. — Boston, MA : Addison-Wesley, 2002.

2. Model-Based Trace-Checking / Y. Howard [и др.] // arXiv preprint arXiv:1111.2825. — 2011. — Нояб. — arXiv: 1111.2825 [cs.SE].

3. How Amazon Web Services Uses Formal Methods / C. Newcombe [и др.] // Communications of the ACM. — 2015. — Март. — Т. 58, № 4. — С. 66—73. — DOI: 10.1145/2699417.

4. Distributed Systems: Concepts and Design / G. Coulouris [и др.]. — 5-е изд. — Addison-Wesley/Pearson, 2012. — ISBN 9780137521081. — URL: https://www.pearson.com/en-us/subject-catalog/p/distributed-systems-concepts-and-design/P200000003160/9780137521081.

5. Ongaro, Diego, and John Ousterhout. In search of an understandable consensus algorithm // In Proceedings of the 2014 USENIX conference on USENIX Annual Technical Conference. — 2006. — С. 305—320.

6. Lynch N. Distributed Algorithms. — Morgan Kaufmann, 1996. — ISBN 9781558603486. — URL: https://shop.elsevier.com/books/distributed-algorithms/lynch/978-1-55860-348-6.

ПРИЛОЖЕНИЕ А

Методика верификации

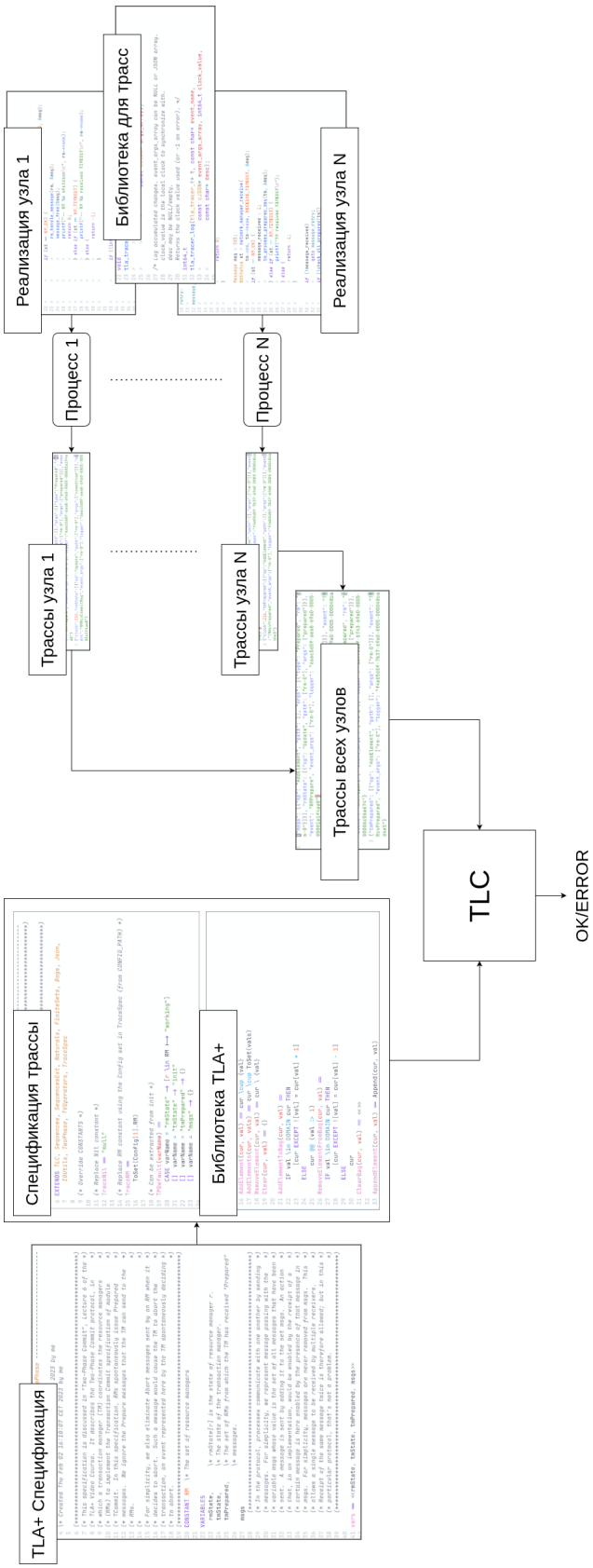

Рисунок А.1 – Методика верификации

ПРИЛОЖЕНИЕ Б

Спецификация Raft на TLA+

В данном приложении приведён полный текст TLA+-спецификации алгоритма консенсуса Raft, использованной в работе для модельной проверки и сопоставления с трассами исполнения реализации. Спецификация описывает состояния узлов, обмен сообщениями, правила выборов лидера, репликации журнала и проверяемые инварианты безопасности.

Листинг Б.1 – Реализация Raft

```
-------------------------------- MODULE Raft --------------------------------

EXTENDS Naturals, FiniteSets, Sequences, TLC

CONSTANTS Server
CONSTANTS Value
CONSTANTS Follower, Candidate, Leader
CONSTANTS Nil
CONSTANTS RequestVoteRequest, RequestVoteResponse,
AppendEntriesRequest, AppendEntriesResponse
CONSTANTS EqualTerm, LessOrEqualTerm
CONSTANTS MaxElections, MaxRestarts

----
\* Global variables
VARIABLE messages
VARIABLE acked
VARIABLE electionCtr, restartCtr
auxVars == <<acked, electionCtr, restartCtr>>
----
\* Per server variables (functions with domain Server).
VARIABLE currentTerm
VARIABLE state
VARIABLE votedFor
serverVars == <<currentTerm, state, votedFor>>

VARIABLE log
VARIABLE commitIndex
logVars == <<log, commitIndex>>

VARIABLE votesGranted
candidateVars == <<votesGranted>>

VARIABLE nextIndex
VARIABLE matchIndex
VARIABLE pendingResponse
leaderVars == <<nextIndex, matchIndex, pendingResponse>>
----
\* All variables; used for stuttering (asserting state hasn't changed).
vars == <<messages, serverVars, candidateVars, leaderVars, logVars, auxVars>>
view == <<messages, serverVars, candidateVars, leaderVars, logVars >>
symmServers == Permutations(Server)
symmValues == Permutations(Value)
----

\* Helpers
Quorum == {i \in SUBSET(Server) : Cardinality(i) * 2 > Cardinality(Server)}

LastTerm(xlog) == IF Len(xlog) = 0 THEN 0 ELSE xlog[Len(xlog)].term

_SendNoRestriction(m) ==
```

```
IF m \in DOMAIN messages
THEN messages' = [messages EXCEPT ![m] = @ + 1]
ELSE messages' = messages @@ (m :> 1)

_SendOnce(m) ==
/\ m \notin DOMAIN messages
/\ messages' = messages @@ (m :> 1)

Send(m) ==
IF /\ m.mtype = AppendEntriesRequest
/\ m.mentries = <<>>
THEN _SendOnce(m)
ELSE _SendNoRestriction(m)

SendMultipleOnce(msgs) ==
/\ \A m \in msgs : m \notin DOMAIN messages
/\ messages' = messages @@ [msg \in msgs |-> 1]

Duplicate(m) ==
/\ m \in DOMAIN messages
/\ messages' = [messages EXCEPT ![m] = @ + 1]

Discard(m) ==
/\ m \in DOMAIN messages
/\ messages[m] > 0 \* message must exist
/\ messages' = [messages EXCEPT ![m] = @ - 1]

Reply(response, request) ==
/\ messages[request] > 0 \* request must exist
/\ \/ /\ response \notin DOMAIN messages
/\ messages' = [messages EXCEPT ![request] = @ - 1] @@ (response :> 1)
\/ /\ response \in DOMAIN messages
/\ messages' = [messages EXCEPT ![request] = @ - 1,
![response] = @ + 1]

ReceivableMessage(m, mtype, term_match) ==
/\ messages[m] > 0
/\ m.mtype = mtype
/\ \/ /\ term_match = EqualTerm
/\ m.mterm = currentTerm[m.mdest]
\/ /\ term_match = LessOrEqualTerm
/\ m.mterm <= currentTerm[m.mdest]

Min(s) == CHOOSE x \in s : \A y \in s : x <= y
Max(s) == CHOOSE x \in s : \A y \in s : x >= y

----
\* Define initial values for all variables
InitServerVars == /\ currentTerm = [i \in Server |-> 1]
/\ state       = [i \in Server |-> Follower]
/\ votedFor    = [i \in Server |-> Nil]
InitCandidateVars == votesGranted   = [i \in Server |-> {}]
InitLeaderVars == /\ nextIndex  = [i \in Server |-> [j \in Server |-> 1]]
/\ matchIndex = [i \in Server |-> [j \in Server |-> 0]]
InitLogVars == /\ log             = [i \in Server |-> << >>]
/\ commitIndex     = [i \in Server |-> 0]
/\ pendingResponse = [i \in Server |-> [j \in Server |-> FALSE]]
InitAuxVars == /\ electionCtr = 0
/\ restartCtr = 0
/\ acked = [v \in Value |-> Nil]

Init == /\ messages = [m \in {} |-> 0]
/\ InitServerVars
/\ InitCandidateVars
/\ InitLeaderVars
/\ InitLogVars
/\ InitAuxVars

----
```

```
\* Define state transitions
Restart(i) ==
/\ restartCtr < MaxRestarts
/\ state'           = [state EXCEPT ![i] = Follower]
/\ votesGranted'    = [votesGranted EXCEPT ![i] = {}]
/\ nextIndex'       = [nextIndex EXCEPT ![i] = [j \in Server |-> 1]]
/\ matchIndex'      = [matchIndex EXCEPT ![i] = [j \in Server |-> 0]]
/\ pendingResponse' = [pendingResponse EXCEPT ![i] = [j \in Server |-> FALSE]]
/\ commitIndex'     = [commitIndex EXCEPT ![i] = 0]
/\ restartCtr'      = restartCtr + 1
/\ UNCHANGED <<messages, currentTerm, votedFor, log, acked, electionCtr>>

RequestVote(i) ==
/\ electionCtr < MaxElections
/\ state[i] \in {Follower, Candidate}
/\ state' = [state EXCEPT ![i] = Candidate]
/\ currentTerm' = [currentTerm EXCEPT ![i] = currentTerm[i] + 1]
/\ votedFor' = [votedFor EXCEPT ![i] = i] \* votes for itself
/\ votesGranted'   = [votesGranted EXCEPT ![i] = {i}] \* votes for itself
/\ electionCtr' = electionCtr + 1
/\ SendMultipleOnce(
{[mtype         |-> RequestVoteRequest,
mterm         |-> currentTerm[i] + 1,
mlastLogTerm  |-> LastTerm(log[i]),
mlastLogIndex |-> Len(log[i]),
msource       |-> i,
mdest         |-> j] : j \in Server \ {i}})
/\ UNCHANGED <<acked, leaderVars, logVars, restartCtr>>

AppendEntries(i, j) ==
/\ i /= j
/\ state[i] = Leader
/\ pendingResponse[i][j] = FALSE \* not already waiting for a response
/\ LET prevLogIndex == nextIndex[i][j] - 1
prevLogTerm == IF prevLogIndex > 0 THEN
log[i][prevLogIndex].term
ELSE
0
\* Send up to 1 entry, constrained by the end of the log.
lastEntry == Min({Len(log[i]), nextIndex[i][j]})
entries == SubSeq(log[i], nextIndex[i][j], lastEntry)
IN
/\ pendingResponse' = [pendingResponse EXCEPT ![i][j] = TRUE]
/\ Send([mtype          |-> AppendEntriesRequest,
mterm          |-> currentTerm[i],
mprevLogIndex  |-> prevLogIndex,
mprevLogTerm   |-> prevLogTerm,
mentries       |-> entries,
mcommitIndex   |-> Min({commitIndex[i], lastEntry}),
msource        |-> i,
mdest          |-> j])
/\ UNCHANGED <<serverVars, candidateVars, nextIndex, matchIndex, logVars, auxVars>>

BecomeLeader(i) ==
/\ state[i] = Candidate
/\ votesGranted[i] \in Quorum
/\ state'      = [state EXCEPT ![i] = Leader]
/\ nextIndex'  = [nextIndex EXCEPT ![i] =
[j \in Server |-> Len(log[i]) + 1]]
/\ matchIndex' = [matchIndex EXCEPT ![i] =
[j \in Server |-> 0]]
/\ pendingResponse' = [pendingResponse EXCEPT ![i] =
[j \in Server |-> FALSE]]
/\ UNCHANGED <<messages, currentTerm, votedFor, candidateVars,
auxVars, logVars>>

AdvanceCommitIndex(i) ==
/\ state[i] = Leader
/\ LET \* The set of servers that agree up through index.
```

```
Agree(index) == {i} \cup {k \in Server :
/\ matchIndex[i][k] >= index }
\* The maximum indexes for which a quorum agrees
agreeIndexes == {index \in 1..Len(log[i]) :
Agree(index) \in Quorum}
\* New value for commitIndex'[i]
newCommitIndex ==
IF /\ agreeIndexes /= {}
/\ log[i][Max(agreeIndexes)].term = currentTerm[i]
THEN
Max(agreeIndexes)
ELSE
commitIndex[i]
IN
/\ commitIndex[i] < newCommitIndex \* only enabled if it actually advances
/\ commitIndex' = [commitIndex EXCEPT ![i] = newCommitIndex]
/\ acked' = [v \in Value |->
IF acked[v] = FALSE
THEN v \in { log[i][index].value : index \in commitIndex[i]+1..newCommitIndex }
ELSE acked[v]]
/\ UNCHANGED <<messages, serverVars, candidateVars, leaderVars, log,
pendingResponse, electionCtr, restartCtr>>

UpdateTerm ==
\E m \in DOMAIN messages :
/\ m.mterm > currentTerm[m.mdest]
/\ currentTerm'    = [currentTerm EXCEPT ![m.mdest] = m.mterm]
/\ state'          = [state       EXCEPT ![m.mdest] = Follower]
/\ votedFor'       = [votedFor    EXCEPT ![m.mdest] = Nil]
\* messages is unchanged so m can be processed further.
/\ UNCHANGED <<messages, candidateVars, leaderVars, logVars, auxVars>>

HandleRequestVoteRequest ==
\E m \in DOMAIN messages :
/\ ReceivableMessage(m, RequestVoteRequest, LessOrEqualTerm)
/\ LET i     == m.mdest
j     == m.msource
logOk == \/ m.mlastLogTerm > LastTerm(log[i])
\/ /\ m.mlastLogTerm = LastTerm(log[i])
/\ m.mlastLogIndex >= Len(log[i])
grant == /\ m.mterm = currentTerm[i]
/\ logOk
/\ votedFor[i] \in {Nil, j}
IN /\ m.mterm <= currentTerm[i]
/\ \/ grant  /\ votedFor' = [votedFor EXCEPT ![i] = j]
\/ ~grant /\ UNCHANGED votedFor
/\ Reply([mtype        |-> RequestVoteResponse,
mterm        |-> currentTerm[i],
mvoteGranted |-> grant,
msource      |-> i,
mdest        |-> j],
m)
/\ UNCHANGED <<state, currentTerm, candidateVars, leaderVars,
logVars, auxVars>>

HandleRequestVoteResponse ==
\E m \in DOMAIN messages :
\* This tallies votes even when the current state is not Candidate, but
\* they won't be looked at, so it doesn't matter.
/\ ReceivableMessage(m, RequestVoteResponse, EqualTerm)
/\ LET i     == m.mdest
j     == m.msource
IN
/\ \/ /\ m.mvoteGranted
/\ votesGranted' = [votesGranted EXCEPT ![i] =
votesGranted[i] \cup {j}]
\/ /\ ~m.mvoteGranted
/\ UNCHANGED <<votesGranted>>
/\ Discard(m)
```

```
/\ UNCHANGED <<serverVars, votedFor, leaderVars, logVars,
auxVars>>
LogOk(i, m) ==
\/ m.mprevLogIndex = 0
\/ /\ m.mprevLogIndex > 0
/\ m.mprevLogIndex <= Len(log[i])
/\ m.mprevLogTerm = log[i][m.mprevLogIndex].term

RejectAppendEntriesRequest ==
\E m \in DOMAIN messages :
/\ ReceivableMessage(m, AppendEntriesRequest, LessOrEqualTerm)
/\ LET i     == m.mdest
j     == m.msource
logOk == LogOk(i, m)
IN  /\ \/ m.mterm < currentTerm[i]
\/ /\ m.mterm = currentTerm[i]
/\ state[i] = Follower
/\ \lnot logOk
/\ Reply([mtype           |-> AppendEntriesResponse,
mterm           |-> currentTerm[i],
msuccess        |-> FALSE,
mmatchIndex     |-> 0,
msource         |-> i,
mdest           |-> j],
m)
/\ UNCHANGED <<state, candidateVars, leaderVars, serverVars,
logVars, auxVars>>

CanAppend(m, i) ==
/\ m.mentries /= << >>
/\ Len(log[i]) = m.mprevLogIndex

NeedsTruncation(m, i, index) ==
\/ /\ m.mentries /= <<>>
/\ Len(log[i]) >= index
\/ /\ m.mentries = <<>>
/\ Len(log[i]) > m.mprevLogIndex

TruncateLog(m, i) ==
[index \in 1..m.mprevLogIndex |-> log[i][index]]

AcceptAppendEntriesRequest ==
\E m \in DOMAIN messages :
/\ ReceivableMessage(m, AppendEntriesRequest, EqualTerm)
/\ LET i     == m.mdest
j     == m.msource
logOk == LogOk(i, m)
index == m.mprevLogIndex + 1
IN
/\ state[i] \in { Follower, Candidate }
/\ logOk
/\ LET new_log == CASE CanAppend(m, i) ->
[log EXCEPT ![i] = Append(log[i], m.mentries[1])]
[] NeedsTruncation(m, i , index) /\ m.mentries # <<>> ->
[log EXCEPT ![i] = Append(TruncateLog(m, i), m.mentries[1])]
[] NeedsTruncation(m, i , index) /\ m.mentries = <<>> ->
[log EXCEPT ![i] = TruncateLog(m, i)]
[] OTHER -> log
IN
/\ state' = [state EXCEPT ![i] = Follower]
/\ commitIndex' = [commitIndex EXCEPT ![i] =
m.mcommitIndex]
/\ log' = new_log
/\ Reply([mtype           |-> AppendEntriesResponse,
mterm           |-> currentTerm[i],
msuccess        |-> TRUE,
mmatchIndex     |-> m.mprevLogIndex +
Len(m.mentries),
msource         |-> i,
```

```
mdest           |-> j],
m)
/\ UNCHANGED <<candidateVars, leaderVars, votedFor, currentTerm,
auxVars>>

HandleAppendEntriesResponse ==
\E m \in DOMAIN messages :
/\ ReceivableMessage(m, AppendEntriesResponse, EqualTerm)
/\ LET i     == m.mdest
j     == m.msource
IN
/\ \/ /\ m.msuccess \* successful
/\ nextIndex'  = [nextIndex  EXCEPT ![i][j] = m.mmatchIndex + 1]
/\ matchIndex' = [matchIndex EXCEPT ![i][j] = m.mmatchIndex]
\/ /\ \lnot m.msuccess \* not successful
/\ nextIndex' = [nextIndex EXCEPT ![i][j] =
Max({nextIndex[i][j] - 1, 1})]
/\ UNCHANGED <<matchIndex>>
/\ pendingResponse' = [pendingResponse EXCEPT ![i][j] = FALSE]
/\ Discard(m)
/\ UNCHANGED <<serverVars, candidateVars, logVars, auxVars>>

----
\* Defines how the variables may transition.
Next ==
\/ \E i \in Server : Restart(i)
\/ \E i \in Server : RequestVote(i)
\/ \E i \in Server : BecomeLeader(i)
\/ \E i \in Server : AdvanceCommitIndex(i)
\/ \E i,j \in Server : AppendEntries(i, j)
\/ UpdateTerm
\/ HandleRequestVoteRequest
\/ HandleRequestVoteResponse
\/ RejectAppendEntriesRequest
\/ AcceptAppendEntriesRequest
\/ HandleAppendEntriesResponse

NoStuttering ==
WF_vars(Next)

Spec == Init /\ [][Next]_vars

LivenessSpec == Init /\ [][Next]_vars /\ NoStuttering

\* INVARIANTS -------------------------
MinCommitIndex(s1, s2) ==
IF commitIndex[s1] < commitIndex[s2]
THEN commitIndex[s1]
ELSE commitIndex[s2]

NoLogDivergence ==
\A s1, s2 \in Server :
IF s1 = s2
THEN TRUE
ELSE
LET lowest_common_ci == MinCommitIndex(s1, s2)
IN IF lowest_common_ci > 0
THEN \A index \in 1..lowest_common_ci : log[s1][index] = log[s2][index]
ELSE TRUE

LeaderHasAllAckedValues ==
\* for every acknowledged value
\A v \in Value :
IF acked[v] = TRUE
THEN
\* there does not exist a server that
~\E i \in Server :
\* is a leader
/\ state[i] = Leader
```

```
\* and which is the newest leader (aka not stale)
/\ ~\E l \in Server :
/\ l # i
/\ currentTerm[l] > currentTerm[i]
\* and that is missing the value
/\ ~\E index \in DOMAIN log[i] :
log[i][index].value = v
ELSE TRUE

==============================================================================
```